



CS & IT ENGINEERING



Algorithms

Analysis of Algorithms

Lecture No.- 08



By- Aditya sir

Recap of Previous Lecture



Topic

Topic

Imp Questions

PYQs.

Topics to be Covered



Topic

Topic

Topic

Analysis of Algo



About Aditya Jain sir

1. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt - City topper
2. Represented college as the first Google DSC Ambassador.
3. The only student from the batch to secure an internship at Amazon. (9+ CGPA)
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program, specialization in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis
8. Completed my Masters with an overall GPA of 9.36/10
9. Joined Dream11 as a Data Scientist
10. Have mentored 12,000+ students & working professions in field of Data Science and Analytics
11. Have been mentoring & teaching GATE aspirants to secure a great rank in limited time
12. Have got around 27.5K followers on LinkedIn where I share my insights and guide students and professionals.



Telegram Link for Aditya Jain sir: https://t.me/AdityaSir_PW

TTTe

[MCQ]

#Q. Arrange the given functions in increasing order of rate of growth:

$$f_1 \rightarrow 2^n, f_2 \rightarrow n^{3/2}, f_3 \rightarrow n \log n, f_4 \rightarrow n^{(\log n)}$$

54%

A

f_2, f_3, f_1, f_4 X

B

f_3, f_2, f_1, f_4 X

C

f_3, f_2, f_4, f_1 ✓

D

f_2, f_3, f_4, f_1 X

$$n^{1.5} < n^{\log n}$$

$$f_3 < (f_2 < f_4) < f_1$$

- 1) 2^n
- 2) $n^{3/2}$
- 3) $n \log n$
- 4) $n^{(\log n)}$

$$f_2 > f_3$$

$$f_1 > f_4$$

$$n^{3/2} \quad n \log n$$

$$n^{1.5}$$

$$n \times \sqrt{n} \quad n \log n$$

$$\sqrt{n} > \log n$$

$$2^n > n^{\log n}$$

$$\log_2(2^n) \quad \log_2(n^{\log n})$$

$$n \times \log_2 2 \quad \log n \times \log n$$

$$n > (\log n)^2$$

[NAT]



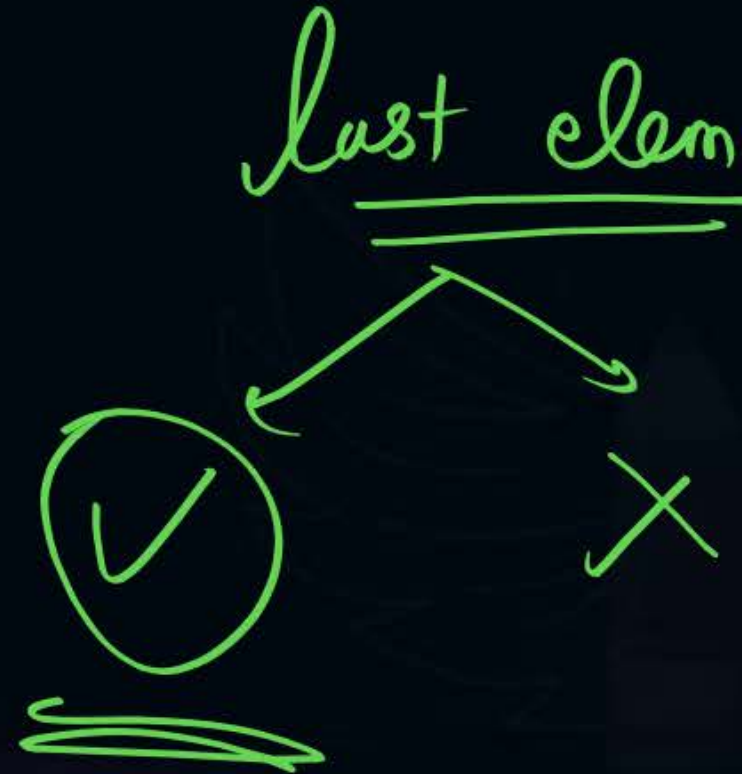
V. Jomp

#Q. An element in an Array is called Leader if it is greater than all elements to the right of it. The time complexity of the most efficient algorithm to print all Leaders of the given Array of size 'n' is _____.

1) Brute Force ✓

2) optimised

take

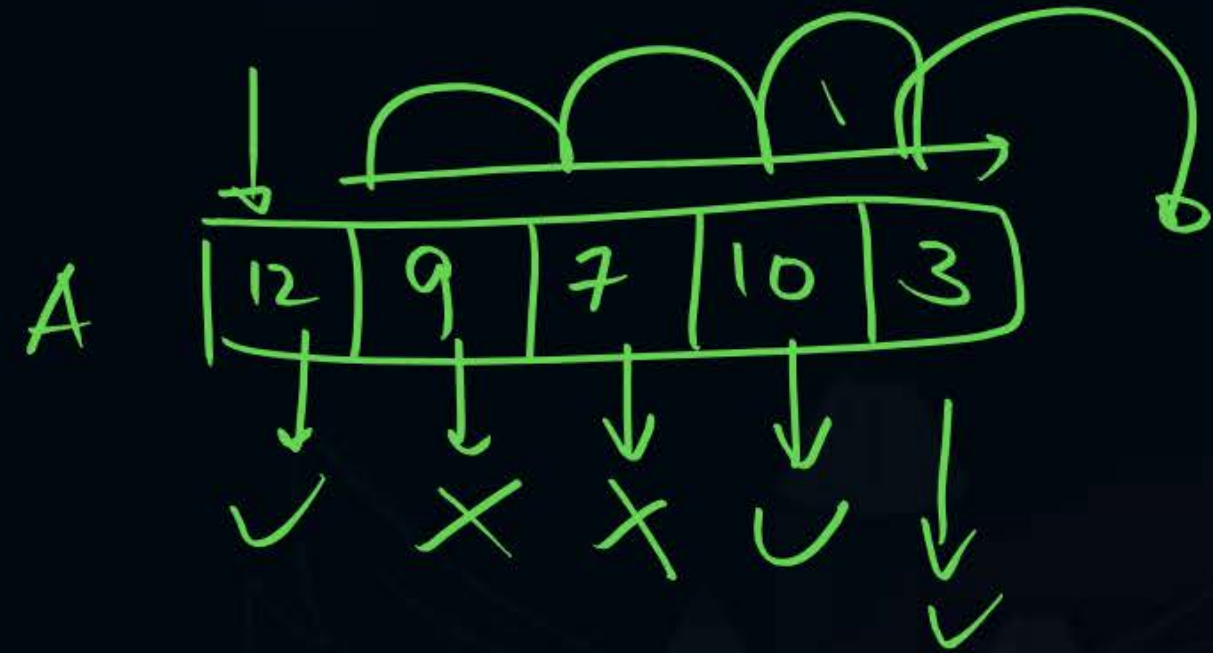


Solution 1: Brute Force

```

Algo ALeader(A[], n) {
    for (i = 1; i < n; i++) {
        for (j = i+1; j ≤ n; j++) {
            if (A[i] < A[j]) {
                break
            }
        }
        ⇒ if (j == (n+1)) {
            print(A[i])
        }
    }
}
    
```

1 based indexing



leaders: 12, 10, 3



Topic : Analysis of Algorithms

Time Complexity Analysis of Brute Force Approach:

(1) Best Case:

A is in increasing order.

E.g.: A = 5, 10, 12, 18

Total Comparisons $\rightarrow \underline{(n-1)} * \underline{O(1)} = O(n-1)$

$\rightarrow \underline{O(n)}$ $\Omega(n)$

1 leaders



Topic : Analysis of Algorithms

Time Complexity Analysis of Brute Force Approach:

(2) Worst Case:

A is in decreasing order.

E.g.: A = 18, 15, 12, 10, 5

Total Comparisons $\rightarrow 4 + 3 + 2 + 1$

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$$

In general $\rightarrow (n-1) + (n-2) + (n-3) \dots 1 = \frac{n(n+1)}{2} - n$

$$= \frac{n^2 - n}{2} \rightarrow O(n^2)$$

Logic:

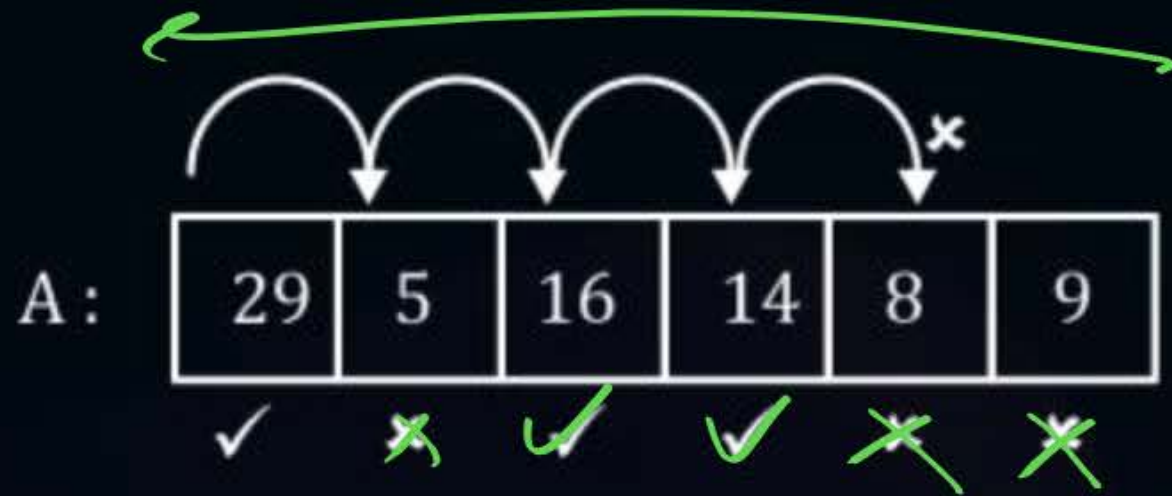
$e > \text{all right} \implies$

$e > \text{max elem on right}$



Topic : Analysis of Algorithms

Code walk through



Handwritten notes showing comparisons and updates to the leader value L:

- $14 > 9 \rightarrow 14$
- $16 > 14 \rightarrow 16$
- $29 > 16 \rightarrow 29$

Algo. A] Leader (a, n)

```
{
  initial ← L = A[n] → O(1)
  O(n) ← for (i = (n - 1); i >= 1; i--)
  {
    if (A[i] > L)
    {
      L = A[i]
      print (A[i])
    }
  }
```




Topic : Analysis of Algorithms

Time complexity Analysis

1. Best Case:- ~~Increasing order~~ $\rightarrow O(n) \rightarrow \underline{\Omega(n)}$
 2. Worst Case:- ~~Decreasing order~~ $\rightarrow O(n) \rightarrow \underline{O(n)}$
- } $\Theta(n)$

Imp. Irrespective of the input order the Algo will take same amount time $\rightarrow \underline{\Theta(n)}$

TC $\Rightarrow$ efficient ~~Ent~~ algo $\Theta(n)$



Topic : Analysis of Algorithms

Summary

10 - 50M

	Best Case	<u>Worst Case</u>
1. Brute force	$O(n)$	$O(n^2)$
2. Efficient Algo	$O(n)$	<u>$O(n)$</u>

→
better



Topic : Analysis of Algorithms

#Q. Arrange the following function in increasing order of their growth rate?

$$f_1 \rightarrow n \log n, \quad f_2 \rightarrow n^2, \quad f_3 \rightarrow e^n, \quad f_4 \rightarrow n^{3/2}, \quad f_5 \rightarrow n, \quad f_6 \rightarrow (1/n), \quad f_7 \rightarrow 2^n$$

- A** $F_2, F_1, F_4, F_3, F_5, F_7, F_6$ ✗
- B** $F_6, F_5, F_1, F_4, F_2, F_3, F_7$ ✗
- C** $F_6, F_5, F_1, F_4, F_2, F_7, F_3$ ✓
- D** $F_6, F_5, F_1, F_4, F_7, F_3, F_2$ ✗

©✓

95%

66%

$$f_1 = n \log n \checkmark$$

$$f_2 = n^2 \checkmark$$

$$f_3 = e^n \checkmark$$

$$f_4 = n^{3/2} \checkmark$$

$$f_5 = n \checkmark$$

$$f_6 = (1/n) \rightarrow \text{smallest}$$

$$f_7 = 2^n \checkmark$$

$$2^n < e^n$$

$$2^n < (2.71)^n$$

$$f_7 < f_3$$

$$n < n^{1.5} < n^2$$

$$n < n \log n < n^{1.5} < n^2$$

$$f_6 < f_5 < f_1 < f_4 < f_2 < f_7 < f_3$$



Topic : Analysis of Algorithms

#Q. Arrange in decreasing order of growth.

$$f_1 \rightarrow n^{1/3}, n (\log n)^{10}, n^{7/4}, (2.002)^n, f_5 \rightarrow e^n$$

90%
↓
72.5%

A $F_5 > F_4 > F_1 > F_3 > F_2$ ✗

B $F_4 > F_5 > F_2 > F_3 > F_1$ ✗

C $F_4 > F_5 > F_3 > F_2 > F_1$ ✗

D $F_5 > F_4 > \underline{\underline{F_3 > F_2 > F_1}}$

$$f_1 = n^{1/3}$$

$$f_2 = n(\log n)^{10}$$

$$f_3 = n^{7/4} \approx n^{1.75}$$

$$f_4 = (2.002)^n$$

$$f_5 = e^n$$

$$f_1 < f_2$$

$$n^{1/3} < n^{7/4}$$

$$f_1 < f_3$$

$$2.002 < e$$

$$(2.002)^n < e^n$$

$$\underline{f_4 < f_5}$$

$$f_3 > f_2$$

$$\underline{n^{1/3}} \quad \underline{n(\log n)^{10}}$$

$$\boxed{n^{1/3} < n}$$

$$n^{1/3} < \underline{n(\log n)^{10}}$$

$$n^{7/4} > n(\log n)^{10}$$

$$n^{1+3/4} \quad n(\log n)^{10}$$

$$\cancel{n \times n^{3/4}} \quad \cancel{n(\log n)^{10}}$$

$$n^{3/4} \quad (\log n)^{10}$$

$$\frac{3}{4} \log n > 10 \log(\log n)$$

$$\frac{3}{4} x > 10 \log(x)$$

Update

Fri

Mon Tu

20-24th June

→ No Algo lecture

↓
Revision



Topic : Analysis of Algorithms

Frame work to determine the time complexity of ~~recurring~~ algorithm.

Recursive



Topic : Analysis of Algorithms

Recursion:

$$n! = n * (n-1) * (n-2) \dots 1$$

$$5! = f(5) = 5 * f(4)$$

↓ 24

$$4 * f(3)$$

↓ 6

$$3 * f(2)$$

↓ 2

$$2 * f(1)$$

↓

$$5 \times 24$$

$$= 120$$

$$1! = 1$$

B.C

Base condition / terminating condition $f(1) = 1$

Recursion stack

f(1)	✓
f(2)	✓
f(3)	✓
f(4)	✓
f(5)	✓



Topic : Analysis of Algorithms

Framework to determine TC of recursive algo.

3 Steps:-

TC Recurrence.

Step-1. Find (Recursive equation for TC) for the recursive code.

Step-2. Solve this Recursive equation to get a mathematical expression/value

Step-2. Apply Asymptotic notation on this value.



Topic : Analysis of Algorithms

TC Recursive equation

(this chapter)

1. Back substitution method

➤ General approach

➤ Gives

- Values of recurrence
- Also give TC → O, Θ, Ω

(DnC)

2. Master Method/ Theorem

Work in specific case

- Only gives final TC but not value

→ Shortcut

Recurrence Tree apply

3. ~~Recursive Tree Approach~~

TC(Final but no. value)

⇒ Asymmetric Recurrence
⇒ (last chapter)



Topic : Analysis of Algorithms

Example:-

Recursive ✓

Algo f(n)

{

if (n==1)

return 1

C1

return $n * f(n-1)$

}

$T(n-1)$



$n!$

$$f(n) = n * f(n-1)$$



Topic : Analysis of Algorithms

Let $T(n)$ Represent the time complexity of $f(n)$

$$T(n) = T(n-1) + \underline{C}, \text{ ~~n > 1~~ } n > 1$$

$$T(n) = C1, n = 1$$

↓
Form Const

Step 1 :- Recursion Code
↓
TC Recurrence



Summary



→ Tc analysis of an Algo

- 1) Brute Force
- 2) Optimised.

→ framework for Tc of Recursive Algo.



THANK - YOU